

Job Portal Full Architecture Refactor (Professional Structure)

FINAL PROJECT FLOW

Recruiter

- Register
- Login
- Add Jobs
- Edit Own Jobs
- Delete Own Jobs
- View Only Own Jobs
- View Applications For Own Jobs

Candidate

- Register
- Login
- View All Jobs
- Apply Jobs
- Upload Resume
- View Own Applications

FINAL DATABASE STRUCTURE

recruiters

Column
id
name
email
password
role
created_at

candidates

Column
id
name
email
password
role
created_at

jobs

Column
id
company_name
title
description
open_date
close_date
recruiter_id

applications

Column
id
applied_date
resume_file
status
job_id
recruiter_id

STEP 1 — FIX USER ARCHITECTURE

User.java

Use ONLY as mapped superclass.

```
package com.jobportal.model;

import jakarta.persistence.*;

@MappedSuperclass
public abstract class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Column(unique = true)
    private String email;

    private String password;

    private String role;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    // getters setters
}
```

IMPORTANT:

- NO @Entity
 - NO users table
-

STEP 2 — Candidate Entity

```
package com.jobportal.model;

import jakarta.persistence.*;

@Entity
@Table(name = "candidates")
public class Candidate extends User {
}
```

STEP 3 — Recruiter Entity

```
package com.jobportal.model;

import jakarta.persistence.*;

@Entity
@Table(name = "recruiters")
public class Recruiter extends User {
}
```

STEP 4 — DELETE OLD FILES

Delete:

- UserRepository.java
- old users table

DO NOT delete User.java.

STEP 5 — CandidateRepository

```
public interface CandidateRepository
extends JpaRepository<Candidate, Long> {

    Optional<Candidate>
```

```
        findByEmail(String email);  
    }
```

STEP 6 — RecruiterRepository

```
public interface RecruiterRepository  
    extends JpaRepository<Recruiter, Long> {  
  
    Optional<Recruiter>  
        findByEmail(String email);  
}
```

STEP 7 — FIX JOB ENTITY

Job.java

```
@Entity  
@Table(name = "jobs")  
public class Job {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private String companyName;  
  
    private String title;  
  
    private String description;  
  
    private LocalDate openDate;  
  
    private LocalDate closeDate;  
  
    @ManyToOne  
    @JoinColumn(name = "recruiter_id")  
    private Recruiter recruiter;  
}
```

IMPORTANT:

- REMOVE candidate_id from jobs table.
-

STEP 8 — FIX APPLICATION ENTITY

Application.java

```
@Entity
@Table(name = "applications")
public class Application {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private LocalDateTime appliedDate;

    private String resumeFile;

    private String status;

    @ManyToOne
    @JoinColumn(name = "job_id")
    private Job job;

    @ManyToOne
    @JoinColumn(name = "candidate_id")
    private Candidate candidate;

    @ManyToOne
    @JoinColumn(name = "recruiter_id")
    private Recruiter recruiter;
}
```

STEP 9 — REGISTER LOGIC

Candidate Registration

Save in candidates table.

Recruiter Registration

Save in recruiters table.

STEP 10 — LOGIN LOGIC

Check role:

Candidate

Search candidate table.

Recruiter

Search recruiter table.

Generate JWT for both.

STEP 11 — FIX CustomUserDetailsService

Search: 1. candidate table 2. recruiter table

Return Spring Security UserDetails.

STEP 12 — JOB ADD FLOW

When recruiter adds job:

```
principal.getName()
```

returns recruiter email.

Find recruiter by email.

Store recruiter object inside job.

STEP 13 — RECRUITER OWN JOBS

Repository:

```
List<Job>  
findByRecruiterId(Long recruiterId);
```

STEP 14 — CANDIDATE SEE ALL JOBS

Use:

```
jobRepository.findAll();
```

STEP 15 — APPLY JOB FLOW

When candidate applies:

1. Get candidate from JWT email
 2. Get job by ID
 3. Get recruiter from job
 4. Save application
-

STEP 16 — APPLICATION SAVE FLOW

Application stores:

- candidate
 - recruiter
 - job
 - resume
 - appliedDate
-

STEP 17 — CANDIDATE APPLICATIONS

Repository:


```
List<Application>
findByCandidateId(Long candidateId);
```

STEP 18 — RECRUITER APPLICATIONS

Repository:

```
List<Application>
findByRecruiterId(Long recruiterId);
```

STEP 19 — FRONTEND ROLE CONDITIONS

Recruiter

Show:

- Add Job
- Edit Job
- Delete Job
- Applicants

Hide:

- Apply Button
 - Resume Upload
-

Candidate

Show:

- Apply Button
- Resume Upload

Hide:

- Edit/Delete
-

STEP 20 — JOBS PAGE LOGIC

Recruiter

```
API.get("/jobs/my-jobs")
```

Candidate

```
API.get("/jobs")
```

STEP 21 — APPLICANTS PAGE

Recruiter

```
API.get("/applications/recruiter")
```

Candidate

```
API.get("/applications/my")
```

STEP 22 — IMPORTANT DATABASE RESET

Because architecture changed heavily:

DROP TABLE:

- applications
- jobs
- candidates
- recruiters
- users

Then:

```
spring.jpa.hibernate.ddl-auto=create
```

Run once.

Then change back:

```
spring.jpa.hibernate.ddl-auto=update
```

FINAL RESULT

You will have:

✔ Professional Recruiter Workflow ✔ Professional Candidate Workflow ✔ Proper Database Normalization
✔ JWT Authentication ✔ Role-Based Access ✔ Recruiter Ownership ✔ Candidate Applications ✔ Resume Upload ✔ CRUD Operations ✔ Real Relational Architecture

This becomes a strong Java Full Stack interview project.